

資訊安全教材 · 入門篇

Web Security 基礎

從 HTTP 原理到常見漏洞觀念的入門教材

教材編製：資安學習系列 | 適用對象：初學者 / CTF 新手

一、為什麼要學 Web Security

網頁應用程式是目前資安攻防最活躍的戰場之一。無論是電商網站、企業內部系統，或是雲端服務的管理後台，幾乎都是以 Web 技術構築而成。理解 Web Security 不只是為了「駭進系統」，更重要的是理解攻擊者的思路，才能設計出真正安全的系統。

學習心法

這份教材著重在「觀念」而非「攻擊武器」。目標是讓你理解每一種漏洞「為什麼會發生」、「攻擊者在利用什麼假設錯誤」，以及「該怎麼防禦」，而不是提供可以直接拿去打正式環境的攻擊指令。

二、HTTP 基礎回顧

2.1 請求與回應

HTTP 是一種無狀態（stateless）的請求-回應協定。用戶端（瀏覽器）送出 Request，伺服器回傳 Response。每一次的請求都包含方法（Method）、路徑（Path）、標頭（Headers）與可能的內容（Body）。

```
GET /profile?id=123 HTTP/1.1
Host: example.com
Cookie: session=abcdef123456
User-Agent: Mozilla/5.0
```

2.2 常見方法

方法	用途	安全性重點
GET	取得資源	參數常出現在 URL，容易被記錄、快取、外洩
POST	提交資料	常用於表單、API，需注意 CSRF 防護
PUT / DELETE	更新 / 刪除資源	常見於 REST API，須有嚴謹的權限控管

2.3 狀態管理：Cookie 與 Session

因為 HTTP 本身無狀態，網站通常透過 Cookie 存放 Session ID，讓伺服器辨識「這是同一個使用者」。這個機制也衍生出許多攻擊面，例如 Session Hijacking（session 被竊取後冒充身份）與 CSRF（利用瀏覽器自動附帶 Cookie 的特性偽造請求）。

三、OWASP Top 10 觀念導覽

OWASP Top 10 是業界公認最具代表性的 Web 弱點分類清單，每幾年會更新一次，反映當前最常見、影響最大的弱點類型。以下挑選幾項最核心的觀念說明。

3.1 Injection（注入類漏洞）

注入類漏洞的核心成因是：**程式將「使用者輸入的資料」與「要執行的指令/查詢語法」混在一起處理**，導致攻擊者可以透過精心設計的輸入，改變程式原本要執行的邏輯。最經典的例子是 SQL Injection。假設後端組合 SQL 字串的方式如下（僅示意，非可執行程式碼）：

```
query = "SELECT * FROM users WHERE username = '" + input_name + "'"
```

如果 `input_name` 沒有經過任何過濾，攻擊者只要輸入含有單引號與 SQL 語法的字串，就可能改變查詢邏輯，繞過登入驗證，甚至讀取不該讀到的資料表。

防禦觀念

- 使用 Prepared Statement / 參數化查詢，讓資料庫明確區分「指令」與「資料」。
- 最小權限原則：資料庫帳號只給該服務真正需要的權限。
- 不要仰賴「黑名單過濾特殊字元」，這類防禦很容易被繞過。

3.2 XSS（Cross-Site Scripting）

XSS 的核心成因與注入類似，但發生在「輸出到瀏覽器」的階段：當網站把使用者輸入的內容，未經適當編碼就直接輸出到 HTML 頁面中，瀏覽器就會把它當成程式碼執行，而不是單純的文字。

--	--

類型	說明
Reflected XSS	惡意腳本包含在請求中（如網址參數），伺服器未過濾就直接反射回頁面
Stored XSS	惡意腳本被儲存在伺服器（如留言板內容），之後每個瀏覽該頁面的人都會受影響
DOM-based XSS	問題發生在前端 JavaScript 處理資料的過程，未經過伺服器

防禦觀念

- 輸出編碼（Output Encoding）：依照輸出情境（HTML、屬性、JS、URL）做對應的跳脫處理。
- 使用現代前端框架（React、Vue）預設的自動跳脫機制，避免手動拼接 HTML 字串。
- 設定 Content-Security-Policy（CSP）標頭，限制可執行的腳本來源，作為深度防禦。

3.3 CSRF（Cross-Site Request Forgery）

CSRF 利用瀏覽器「自動附帶 Cookie」的特性：攻擊者誘導使用者在已登入某網站的狀態下，造訪一個惡意頁面，該頁面偷偷送出對目標網站的請求（例如轉帳、改密碼），瀏覽器會自動帶上使用者的 Cookie，讓伺服器誤以為是使用者本人的操作。

防禦觀念

- CSRF Token：每次表單提交都攜帶一個伺服器產生、與 Session 綁定的隨機 Token。
- SameSite Cookie 屬性：限制 Cookie 是否會在跨站請求中被自動夾帶。
- 對於重要操作，可加入二次驗證（如重新輸入密碼）。

3.4 存取控制失效（Broken Access Control）

這是近年 OWASP Top 10 排名第一的類別。核心問題是：系統只驗證了「你是誰」（Authentication），卻沒有確實驗證「你能不能做這件事 / 存取這筆資料」（Authorization）。

最常見的例子是 IDOR（Insecure Direct Object Reference）：例如網址 `/api/orders/1001` 只要把數字改成 `1002`，就能看到別人的訂單，若後端沒有針對「這筆資料是否屬於目前登入者」做檢查，就會造成資料外洩。

3.5 安全設定錯誤（Security Misconfiguration）

包含：預設帳號未修改、除錯模式（debug mode）在正式環境仍開啟、錯誤訊息洩漏系統內部細節（如完整的堆疊追蹤、資料庫結構）、不必要的服務或連接埠對外開放等。這類問題往往不是程式邏輯錯誤，而是「部署與維運」層面的疏忽。

3.6 加密機制失效（Cryptographic Failures）

常見情境包含：密碼未經雜湊直接明文儲存、使用已知不安全的雜湊演算法（如未加鹽的 MD5）、傳輸敏感資料未使用 HTTPS、自行設計加密演算法而非採用業界驗證過的標準函式庫等。

四、認證與授權的基本觀念

4.1 認證（Authentication）

回答「你是誰」的問題。常見機制包含帳號密碼、多因子驗證（MFA）、OAuth 第三方登入等。密碼儲存應使用專門設計的雜湊演算法（如 bcrypt、argon2），而非一般用途的雜湊函式（如 MD5、SHA1），因為後者運算速度太快，容易被暴力破解或彩虹表攻擊。

4.2 授權（Authorization）

回答「你能不能做這件事」的問題。授權檢查應該在**每一次**存取資源時都由伺服器端重新驗證，絕對不能只依賴前端介面隱藏按鈕，或是相信使用者送來的參數（例如網址中的 `role=admin`）。

五、常用防護機制一覽

機制	作用
HTTPS / TLS	保護傳輸中的資料不被竊聽或竄改
WAF（Web Application Firewall）	在應用層過濾已知攻擊特徵，作為額外加防線
安全標頭（Security Headers）	如 CSP、X-Frame-Options、Strict-Transport-Security 等，強化瀏覽器端防護

輸入驗證（Input Validation）	對格式、長度、型別做白名單式檢查
最小權限原則	帳號、服務、資料庫連線都只給予必要的最小權限

六、學習路徑建議

- 打好 HTTP／瀏覽器同源政策（Same-Origin Policy）基礎：這是理解幾乎所有 Web 漏洞的地基。
- 用合法平台練習：例如 OWASP Juice Shop、DVWA、PortSwigger Web Security Academy 等專為教學設計的靶場。
- 閱讀真實案例分析：CVE 報告、資安研究團隊發布的漏洞分析文章。
- 學習防禦者視角：嘗試自己修補一個有漏洞的範例程式，理解「為什麼這樣改就安全了」。

重要提醒

對任何非自己擁有、且未取得明確書面授權的系統進行滲透測試，都可能觸犯法律（如台灣《刑法》第 358 條妨害電腦使用罪章）。請務必只在合法授權的環境（自建靶場、CTF 競賽、正式簽約的滲透測試專案）中練習與應用這些知識。